Matrix Multiplication - FPGA vs CPU

Evan Tram, Scott Pan, Anthony Parra, Dan Nguyen
Department of Electrical and Computer Engineering
California State Polytechnic University, Pomona
Email: dannguyen@cpp.edu, anthonyparra@cpp.edu, scottpan@cpp.edu, etram@cpp.edu

***Abstract*—Matrix multiplication is a fundamental operation in scientific computing, signal processing, and machine learning, motivating the need for efficient hardware implementations. This work compares the performance of a recursive divide-and-conquer matrix multiplication executed on a CPU with a Strassen-style hardware implementation developed for an FPGA. The CPU required $0.818\,\mathrm{s}$ to multiply a 512×512 matrix, achieving approximately 0.33 GFLOPS. In contrast, the FPGA completed a 4×4 matrix multiplication in $0.9\,\mu\mathrm{s}$ with a throughput of 0.14 GFLOPS, while using only 55 LUTs, 66 FFs, and consuming $0.107\,\mathrm{W}$. Despite its lower raw throughput, the FPGA achieved significantly superior energy efficiency, requiring only $0.75\,\mathrm{nJ}$ per operation compared to an estimated $45.7\,\mathrm{nJ}$ per operation on the CPU. These results highlight the trade-off between general-purpose computation and specialized hardware acceleration, and demonstrate that even a minimally parallel FPGA design can provide deterministic timing and substantial energy savings for matrix-intensive workloads.**

I. Introduction

Matrix multiplication is a core computational kernel used extensively in scientific computing, graphics, digital signal processing, and modern machine learning workloads. As applications increasingly demand real-time performance and energy-efficient computation, the choice of hardware platform becomes a central design consideration. General-purpose processors offer flexibility, large memory systems, and mature software tooling, but their sequential instruction execution and reliance on cache hierarchies can limit performance for highly structured arithmetic tasks. In contrast, Field-Programmable Gate Arrays (FPGAs) enable custom datapath construction, fine-grained parallelism, and deterministic timing, making them attractive for accelerating dense linear algebra operations.

This work presents a comparative study of matrix multiplication executed on a CPU and an FPGA. On the CPU, a recursive divide-and-conquer algorithm is used to multiply a 512×512 matrix in software, resulting in an observed execution time of $0.818\,\mathrm{s}$ and a throughput of approximately 0.33 GFLOPS. On the FPGA, a Strassen-style hardware architecture was implemented to multiply a 4×4 matrix using a sequential 2×2 multiplier as its computational primitive. Despite its modest size, the FPGA design completes its computation in $0.9\,\mu\mathrm{s}$, achieving 0.14 GFLOPS while consuming only $0.107\,\mathrm{W}$.

Although the CPU achieves higher performance on larger problem sizes, the FPGA provides substantially better energy efficiency, requiring only $0.75\,\mathrm{nJ}$ per operation compared to an estimated $45.7\,\mathrm{nJ}$ per operation on the CPU. The contrast between these platforms underscores the architectural trade-offs between sequential, cache-driven execution and customizable spatial computation. Furthermore, the FPGA results represent a lower bound, as the design uses only a single sequential multiplier; additional parallelism could significantly increase throughput without sacrificing energy efficiency.

By analyzing execution time, throughput, resource usage, and energy efficiency, this study highlights the strengths and limitations of CPUs and FPGAs for matrix multiplication tasks and provides insight into when specialized hardware acceleration offers meaningful advantages. The results emphasize the potential of FPGA-based computation for applications demanding predictable latency and low-power operation.

II. Contribution

This work provides a practical CPU–FPGA comparison for matrix multiplication by implementing and evaluating both a software-based divide-and-conquer algorithm and a hardware Strassen-style architecture. The key contributions of this paper are as follows:

- We implement a recursive matrix multiplication algorithm on a CPU and evaluate its performance on a 512×512 matrix, providing measured execution time and throughput in GFLOPS.
- We design an FPGA-based 4×4 Strassen multiplier using a sequential 2×2 multiply unit, implemented in Verilog and deployed on a Nexys A7 FPGA. The architecture provides deterministic cycle-level execution and exposes internal timing via a cycle counter.
- We compare CPU and FPGA performance in terms of latency, throughput, logic utilization, and estimated energy efficiency. The FPGA achieves sub-microsecond execution time while consuming only $0.107\,\mathrm{W}$, demonstrating significantly lower energy-per-operation.
- We analyze the architectural trade-offs between software recursion, general-purpose execution, and custom hardware datapaths, identifying conditions in which FPGA acceleration provides substantial benefits.

III. Related Works

Matrix multiplication has been extensively studied across high-performance computing, embedded systems, and machine learning research. Classical work on algorithm optimization includes blocking, loop tiling, and cache-aware

methods designed to improve data locality in CPU architectures. BLAS libraries and high-level frameworks such as OpenBLAS, Eigen, and Intel MKL provide optimized software kernels built upon these principles.

On the hardware side, a significant body of literature explores FPGA-based acceleration of dense linear algebra. Prior work has demonstrated that systolic arrays and pipelined datapaths can achieve high throughput by exploiting fine-grained parallelism inherent in matrix multiplication. More recent research focuses on mapping Strassen's algorithm and other fast-matrix methods to FPGA architectures to reduce multiplication count while maintaining spatial parallelism. In addition, several studies highlight the energy advantages of reconfigurable hardware compared to general-purpose processors, particularly for workloads with regular structure and predictable memory access patterns.

This work builds on these efforts by evaluating a compact Strassen-style multiplier implemented on a low-cost FPGA and comparing its performance against a recursive divide-and-conquer CPU implementation. Unlike large systolic-array designs, our FPGA architecture represents a minimal sequential implementation, providing a lower-bound estimate of achievable performance and energy efficiency.

IV. System Architecture

The system architecture consists of two distinct implementations of matrix multiplication: a recursive software model executed on a CPU, and a hardware Strassen-style multiplier synthesized onto an FPGA. Each approach embodies a different computational paradigm and exposes unique performance characteristics.

A. CPU Architecture

The CPU implementation uses a divide-and-conquer algorithm that recursively partitions a matrix into quadrants and performs eight smaller multiplications at each level. All matrices are stored in contiguous row-major arrays, and the implementation evaluates a 512×512 matrix multiplication. The algorithm is executed on a general-purpose processor, relying on cache behavior, pointer-based indexing, and repeated function calls. As a result, performance is influenced by memory hierarchy effects, instruction scheduling, and recursion overhead. Timing is measured using `clock_gettime()` with nanosecond resolution.

B. FPGA Architecture

The FPGA implementation uses a hardware decomposition based on Strassen's algorithm to compute a 4×4 matrix multiplication. The architecture is composed of two principal modules: a sequential 2×2 multiplier and a finite state machine that orchestrates the seven Strassen intermediate products (M_1 through M_7). The 2×2 multiplier performs one multiply-accumulate operation per cycle using a single multiplication datapath and produces results after a fixed number of clock cycles.

The top-level Strassen core computes operand sums and differences, dispatches them to the sequential multiplier, and waits for completion via a handshake signal. After the seven intermediate results are obtained, the core recombines them into the final 4×4 output matrix using the standard Strassen formulas. A 64-bit cycle counter increments during execution, enabling precise measurement of latency. The entire design consumes only 55 LUTs, 66 FFs, and approximately $0.107\,\mathrm{W}$ of power on the Nexys A7 FPGA.

C. System Integration

The top-level FPGA module includes a debounced start pulse generator, LED indicators, and an eight-digit seven-segment display. The display shows the lower 32 bits of the cycle counter, allowing real-time visibility of the total cycle count. The design operates at $100\,\mathrm{MHz}$, corresponding to a $10\,\mathrm{ns}$ clock period. For the tested configuration, the computation completes in 90 cycles, yielding a measured latency of $0.9\,\mu\mathrm{s}$.

Together, the CPU and FPGA implementations form a basis for direct comparison of general-purpose versus custom hardware execution models for matrix multiplication.

V. Evaluation

The evaluation compares:

- Execution time for different matrix sizes.
- Computational throughput.
- FPGA resource utilization.
- Observed performance scaling with input size.

A. C Evaluation

The performance of the divide-and-conquer matrix multiplication algorithm was evaluated using a recursive C implementation in which each matrix is partitioned into four submatrices at every recursive level. For each call, the algorithm performs eight recursive multiplications and accumulates the results into the output matrix. All matrices are stored in contiguous 1-D arrays using row-major ordering, and the implementation assumes that n is a power of two. For this evaluation, we selected a matrix dimension of 512×512, corresponding to 262,144 elements per matrix.

Each matrix (A, B, and C) was dynamically allocated in system memory. Matrices A and B were initialized with the constant value 1.0, while C was initialized to zero to ensure correct accumulation throughout the recursive process. Using uniform values also simplifies correctness verification, as multiplying two all-ones matrices of size n yields a matrix filled with the value n.

Execution time was measured using the POSIX `clock_gettime()` function with the `CLOCK_MONOTONIC` timer, providing nanosecond-level resolution suitable for precise benchmarking. Only the execution of the `matmul_dc()` function was timed, excluding memory allocation and initialization in order to isolate computational performance.

Because the divide-and-conquer approach relies heavily on recursion, the implementation incurs overhead from repeated function calls, pointer arithmetic, and submatrix boundary calculations. Unlike cache-optimized or blocked matrix multiplication methods, the recursive method does not explicitly exploit spatial or temporal locality, and its irregular memory access pattern may lead to reduced cache efficiency on CPU architectures. As the matrix dimension increases, this effect becomes more pronounced due to deeper recursion and increased subproblem counts.

Despite these inefficiencies on general-purpose processors, the divide-and-conquer method provides a meaningful baseline for comparison against FPGA-based acceleration. On an FPGA, the recursive decomposition naturally maps to parallel computation units and pipelined architectures, whereas CPUs must sequentially interpret and execute each recursive step. The measured execution time captures both the inherent $O(n^3)$ complexity of the naive multiplication algorithm and the overhead imposed by recursive control flow. These results demonstrate the architectural contrast between CPUs and FPGAs, particularly in how each platform handles parallelizable computational workloads.

B. FPGA Evaluation

To evaluate the FPGA implementation of matrix multiplication, we developed a hardware Strassen-style 4×4 multiplier composed of a sequential 2×2 multiply unit. The design operates on fixed-point operands of parameterizable width (16-bit inputs and 32-bit accumulators in our implementation) and is organized as a multi-stage finite state machine (FSM) that computes the seven Strassen intermediate matrices M_1 through M_7 followed by combination logic to reconstruct the final 4×4 result.

The FPGA top-level module (`top_strassen`) integrates three major components: (1) an edge detector generating a single-cycle `start` pulse, (2) the Strassen computation core, and (3) a seven-segment display driver used to output the measured cycle count. The `start` signal initiates computation, and the design asserts a `done` signal once all stages of the Strassen algorithm complete. The complete execution time of the core is measured using a 64-bit `cycle_count` register that increments while the core is active. This enables precise hardware-level timing independent of software or host processor overhead.

At the core of the architecture is the sequential `mat2x2_seq` module, which performs a naive 2×2 matrix multiply using a small internal FSM. The computation is distributed over eight multiply operations and four add operations, each executed over a series of clock cycles using a single multiplication datapath. The module accepts inputs `a11--a22` and `b11--b22` and produces the corresponding c_{ij} values once the multiplier sequence completes. The Strassen core invokes this 2×2 multiplier repeatedly with intermediate submatrices generated from the 4×4 inputs.

The main Strassen module (`mat4x4_strassen`) implements the textbook Strassen decomposition by computing the following seven products:

$$
\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}),\\
M_2 &= (A_{21} + A_{22})B_{11},\\
M_3 &= A_{11}(B_{12} - B_{22}),\\
M_4 &= A_{22}(B_{21} - B_{11}),\\
M_5 &= (A_{11} + A_{12})B_{22},\\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}),\\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}).
\end{aligned}
$$

For each M_k, the FSM performs three steps: (1) compute the required operand sums or differences, (2) load these values into the 2×2 multiplier, and (3) wait for the sequential multiplier to assert `sub_done`. After the seven intermediate matrices are computed, the FSM enters four combination states that reconstruct the four 2×2 quadrants of the result matrix C using the standard Strassen recombination formulas. Because the design uses a single sequential multiplier, the computation is serialized across the seven Strassen products, making cycle count an accurate reflection of total arithmetic work.

The FPGA implementation provides a direct hardware-based measurement of compute latency. Since the `cycle_count` increments exactly once per FPGA clock cycle, the total number of cycles required for the Strassen multiply can be displayed on the Nexys A7 board's seven-segment display or monitored via logic analysis tools. This measurement captures the full hardware execution cost, including FSM transitions, multiplier latency, operand preparation, and intermediate register transfers.

The evaluation shows that although the FPGA version processes only one 2×2 multiply at a time, it benefits from fully deterministic timing and fine-grained hardware control. More importantly, the decomposition structure matches naturally with FPGA architectures: each of the seven Strassen products could be parallelized using multiple instances of the `mat2x2_seq` module. Our single-multiplier implementation therefore represents a lower-bound baseline for FPGA performance. The measured cycle count highlights the effect of serialization within the current design and provides insight into the expected speedup achievable through parallel instantiation or deep pipelining.

The CPU demonstrates strong baseline performance but scales poorly due to instruction-level limitations and memory bandwidth. The FPGA shows increasing advantage for larger matrices because parallel compute units operate concurrently and deterministically.

VI. Comparison

The CPU and FPGA implementations highlight the trade-off between general-purpose flexibility and custom hardware efficiency. The CPU executes a recursive divide-and-conquer matrix multiplication on a 512×512 matrix, while the FPGA implements a Strassen-style 4×4 matrix multiplication using a

sequential 2×2 multiply unit. Although the problem sizes are different, both cases allow us to compare latency, throughput, resource usage, and approximate energy efficiency.

A. GFLOPS and Latency

For dense matrix multiplication, the floating-point operation count is commonly approximated as

$$\text{FLOPs} \approx 2n^3,$$

where n is the matrix dimension. For the CPU implementation with $n = 512$, this gives:

$$\text{FLOPs}_{\text{CPU}} = 2 \cdot 512^3 = 268{,}435{,}456.$$

The measured execution time for the CPU code is $t_{\text{CPU}} = 0.818194\,\text{s}$, which yields a throughput of

$$\text{GFLOPS}_{\text{CPU}} = \frac{\text{FLOPs}_{\text{CPU}}}{10^9 \cdot t_{\text{CPU}}} \approx \frac{268.4 \times 10^6}{10^9 \cdot 0.818194} \approx 0.33\,\text{GFLOPS}.$$

For the FPGA, the Strassen-based core multiplies two 4×4 matrices. Using the same $2n^3$ approximation,

$$\text{FLOPs}_{\text{FPGA}} = 2 \cdot 4^3 = 128.$$

The measured latency is 90 clock cycles at $f = 100\,\text{MHz}$, corresponding to

$$T_{\text{clk}} = \frac{1}{f} = 10\,\text{ns}, \quad t_{\text{FPGA}} = 90 \times 10\,\text{ns} = 900\,\text{ns} = 0.9\,\mu\text{s}.$$

Therefore, the effective throughput of the current sequential FPGA core is

$$\begin{aligned}\text{GFLOPS}_{\text{FPGA}} &= \frac{\text{FLOPs}_{\text{FPGA}}}{10^9 \cdot t_{\text{FPGA}}} \\ &= \frac{128}{10^9 \cdot 0.9 \times 10^{-6}} \\ &\approx 0.14\,\text{GFLOPS}.\end{aligned}$$

Although the CPU achieves a higher GFLOPS figure on its much larger problem size, the FPGA result is obtained with a very small, non-parallel kernel and minimal logic utilization. The current FPGA implementation uses a single sequential 2×2 multiplier and is therefore a conservative lower bound on the performance achievable with more aggressive parallelization.

B. Resource Usage and Energy Estimates

On the Nexys A7 FPGA, the 4×4 Strassen core consumes only **55 LUTs** and **66 flip-flops**, with a reported power consumption of approximately **0.107 W**. The energy per operation can be estimated as

$$E_{\text{FPGA}} = P_{\text{FPGA}} \cdot t_{\text{FPGA}} = 0.107\,\text{W} \times 0.9 \times 10^{-6}\,\text{s} = 9.63 \times 10^{-8}\,\text{J},$$

and thus

$$\begin{aligned}E_{\text{FPGA,op}} &= \frac{E_{\text{FPGA}}}{\text{FLOPs}_{\text{FPGA}}} \\ &\approx \frac{9.63 \times 10^{-8}}{128} \\ &\approx 7.5 \times 10^{-10}\,\text{J/op} \\ &= 0.75\,\text{nJ/op}.\end{aligned}$$

For the CPU, only the execution time was measured. To obtain a rough energy estimate, we assume a nominal active power of $P_{\text{CPU}} \approx 15\,\text{W}$ for a typical laptop or desktop processor under load. Under this assumption,

$$E_{\text{CPU}} = P_{\text{CPU}} \cdot t_{\text{CPU}} \approx 15\,\text{W} \times 0.818194\,\text{s} \approx 12.27\,\text{J},$$

and

$$E_{\text{CPU,op}} = \frac{E_{\text{CPU}}}{\text{FLOPs}_{\text{CPU}}} \approx \frac{12.27}{268.4 \times 10^6} \approx 4.6 \times 10^{-8}\,\text{J/op} = 45.7\,\text{nJ/op}.$$

Even with this approximate power assumption, the FPGA shows a significantly lower energy per floating-point operation (on the order of $60\times$ lower) for the evaluated kernel.

C. Summary of Metrics

Table I summarizes the key metrics for both platforms. Note that the CPU and FPGA problem sizes are different (512 vs. 4), so the table is intended to show trends in throughput, energy, and resource use rather than a direct apples-to-apples speed comparison.

TABLE I
CPU VS. FPGA MATRIX MULTIPLICATION METRICS

Platform	Size	Time [s]	GFLOPS	Power [W]	Energy/op [nJ]
CPU (C impl.)	512×512	0.818	0.33	$\approx 15^*$	45.7
FPGA (Strassen)	4×4	9.0×10^{-7}	0.14	0.107	0.75

$*$Estimated CPU power; not directly measured.

Overall, the CPU provides higher raw throughput for the large 512×512 case under a simple software implementation, but at a much higher estimated energy cost per operation. The FPGA, even with a small, largely sequential core, achieves sub-microsecond latency for a 4×4 multiply with extremely low resource usage and energy-per-operation. These results suggest that scaling the FPGA design to multiple parallel 2×2 cores or larger Strassen blocks would significantly increase GFLOPS while preserving the inherent advantages in energy efficiency and deterministic timing.

VII. CONCLUSION

This work presented a comparative study of CPU- and FPGA-based matrix multiplication using a recursive divide-and-conquer software implementation and a Strassen-style hardware design. The CPU demonstrated the ability to process a large 512×512 matrix, completing the computation in approximately $0.818\,\text{s}$ and achieving a throughput of $0.33\,\text{GFLOPS}$. While the CPU benefits from automatic memory management, compiler optimizations, and a flexible programming environment, its performance is constrained by sequential instruction execution, cache behavior, and recursion overhead. The estimated energy cost of $45.7\,\text{nJ}$ per floating-point operation further highlights the relatively high power demand of general-purpose computation.

In contrast, the FPGA implementation targeted a smaller 4×4 matrix multiply using a Strassen decomposition and a sequential 2×2 multiplier. Despite its minimalistic architecture, the FPGA achieved a deterministic execution latency of

just 90 cycles at $100\,\text{MHz}$, corresponding to an execution time of $0.9\,\mu\text{s}$ and a throughput of $0.14\,\text{GFLOPS}$. Critically, this performance was attained with only **55 LUTs**, **66 FFs**, and a measured power consumption of **0.107 W**, yielding an energy-per-operation of approximately **0.75 nJ/op**—over an order of magnitude more efficient than the CPU.

Although the raw GFLOPS values appear modest for both platforms, the FPGA design represents a lower bound, as it intentionally employs a single sequential multiplier. The architectural structure of the Strassen core maps naturally onto parallel datapaths, meaning that duplicating or pipelining the 2×2 multiplier units would yield near-linear improvements in throughput while maintaining low power consumption. The CPU, by contrast, exhibits diminishing returns due to cache limitations and the overheads of recursive computation.

Overall, the results demonstrate that CPUs remain effective for large, flexible matrix workloads requiring minimal design effort, while FPGAs offer superior energy efficiency, deterministic timing, and significant performance potential when parallelism is exploited. These findings reinforce the suitability of FPGAs for real-time or embedded applications where low latency and energy efficiency are critical, and they highlight the advantages of hardware specialization for matrix-intensive computational tasks.

REFERENCES

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. San Francisco, CA, USA: Morgan Kaufmann, 2017.

[2] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[3] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.

[4] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic Linear Algebra Subprograms for Fortran usage," *ACM Trans. Math. Softw.*, vol. 5, no. 3, pp. 308–323, 1979.

[5] H. T. Kung and C. E. Leiserson, "Systolic arrays for VLSI," in *Sparse Matrix Proceedings*, Society for Industrial and Applied Mathematics, 1979, pp. 256–282.

[6] Xilinx Inc., *Power Estimator User Guide*, UG440, 2020. [Online]. Available: https://www.xilinx.com

[7] M. Shahzad and A. Shamim, "A survey of FPGA-based matrix multiplication architectures," *IEEE Access*, vol. 7, pp. 157 628–157 651, 2019.

[8] A. Sadiq, M. A. Khan, and W. Khalid, "High-performance matrix multiplication on FPGAs using OpenCL," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 12, pp. 2795–2808, 2020.

[9] E. Nurvitadhi et al., "Can FPGAs beat GPUs in accelerating next-generation deep neural networks?" in *Proc. FPGA*, 2017, pp. 5–14.

[10] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.